



COURSE SLIDES

Building Multi Agent Applications with Gemini ADK (C829)

Course Code: C829

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.0 · 11 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Tertiary Infotech Trainer

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

Industry certified in the subject area of this course.

DELIVERS

Professional short courses for Tertiary Infotech Academy.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

1 Your name and organisation / role.

2 Your experience with this subject (if any).

3 What you want to be able to do after this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

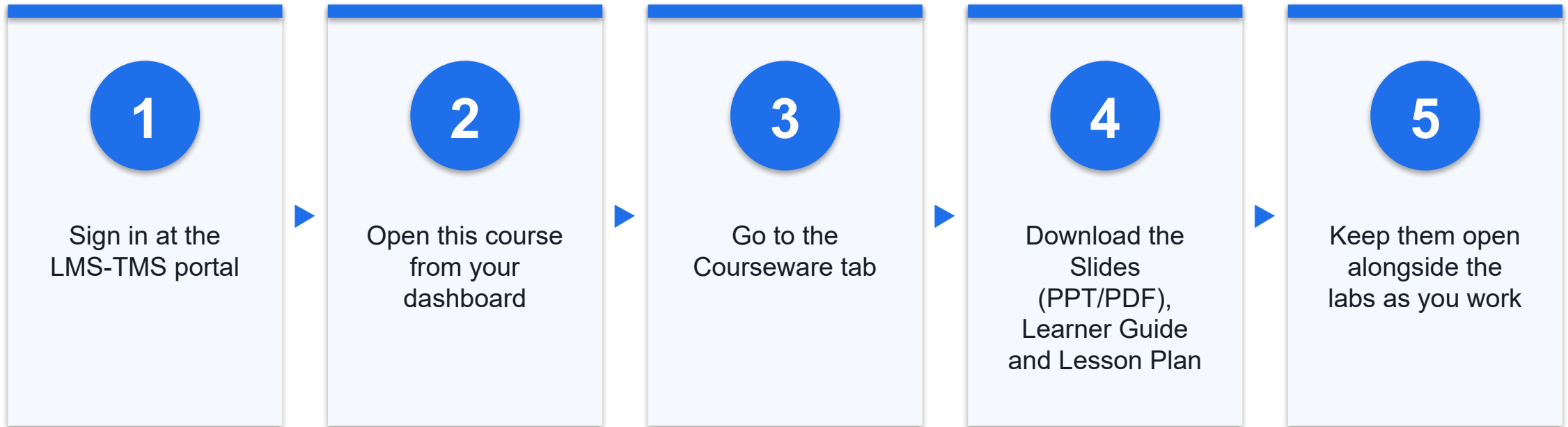
5

Be punctual; return from breaks on time.

6

Learn by doing — try every hands-on lab.

Download Course Material



Portal: <https://lms-tms.tertiaryinfotech.com>

Lesson Plan — 2 Days, 7.5 scheduled hours/day

Day 1

- Day 1 — Single Agents, Tools, State and Memory
 - Topic 1: Single Agent Foundations (Labs 1–2)
 - Topic 2: Tools, Memory and Sessions (Labs 3–4)

Day 2

- Day 2 — Multi-Agent Orchestration, MCP and Production Readiness
 - Topic 3: Multi-Agent Architecture (Labs 5–6)
 - Topic 4: MCP and Production Design (Labs 7–8)
- Daily timing
 - 9:30 am – 6:00 pm (1-hour lunch; tea breaks within scheduled time)

Learning Outcomes

1

Agent Foundations

Explain the ADK agent lifecycle and configure a Gemini agent with clear instructions, typed input or output, and session-aware behaviour.

2

Reliable Tools

Build reliable function tools and API integrations with explicit schemas, validation, error handling, and safe secret management.

3

State & Memory

Use session state and searchable memory to preserve relevant context without confusing conversation history, working state, and long-term knowledge.

4

Agent Teams

Design multi-agent systems with specialist roles, coordinator delegation, shared context, and deterministic graph workflows where control matters.

5

MCP & Routing

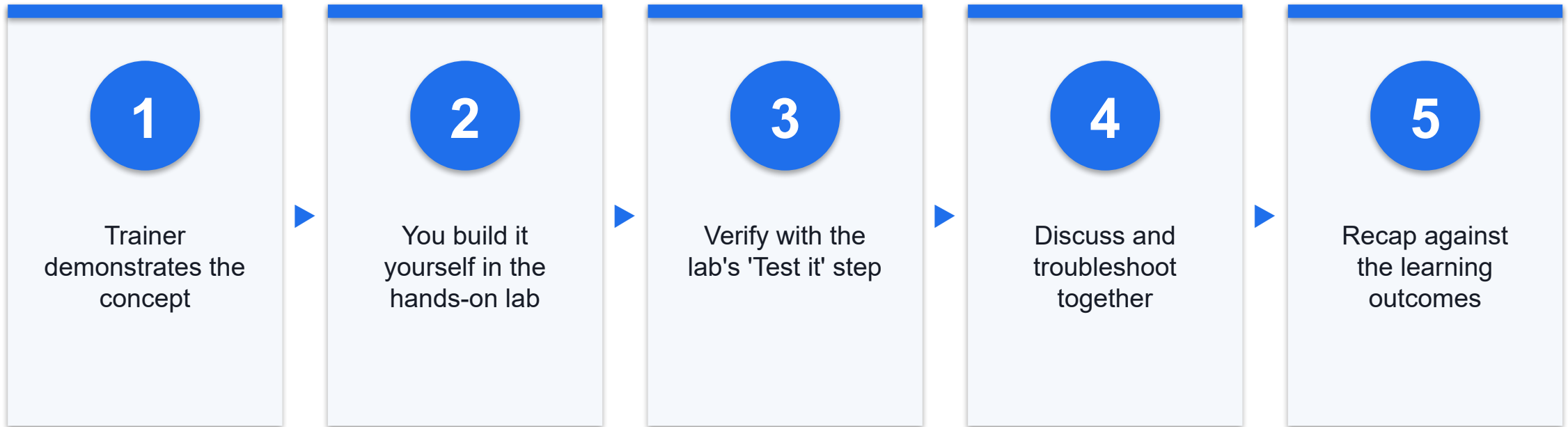
Connect ADK agents to MCP capabilities and choose an appropriate routing strategy for tools, specialists, and workflow branches.

6

Production Readiness

Apply evaluation, observability, security guardrails, and deployment practices to prepare a multi-agent application for production use.

How You'll Learn





CORE CONCEPTS

Course Fundamentals

The ADK System Model

1

Agent

A model-driven component with a role, instructions, tools, schemas, and optional sub-agents.

2

Runner

The application engine that executes the agent and coordinates services across each invocation.

3

Tool

A typed capability that lets an agent read data, calculate, call a service, or perform an approved action.

4

Event

An ordered record of messages, calls, results, state changes, and workflow outputs.

5

Session

One conversation thread, including its events and temporary state.

6

Memory

Searchable knowledge beyond the active thread, often backed by a managed or vector service.

The ADK System Model (cont.)

7

Workflow

An explicit graph or programmatic control flow that combines deterministic logic and agent reasoning.

8

MCP

A standard client-server protocol for discovering and calling external tools and context capabilities.

9

Guardrail

A policy check that observes, blocks, changes, or requires approval around an agent action.

10

Trace

Correlated telemetry that explains the route, model calls, tools, latency, errors, and result.

The DESIGN Production Lens

1

D — Define

Name the user outcome, non-goals, boundaries, and acceptable failure behaviour.

2

E — Encapsulate

Give each agent and tool one clear responsibility with a typed contract.

3

S — Share selectively

Pass only the state, memory, and evidence required for the current task.

4

I — Inspect

Capture events, route decisions, tool calls, traces, cost, and outcome quality.

5

G — Guard

Apply identity, least privilege, validation, approvals, and output controls in layers.

6

N — Normalise

Package, evaluate, version, and deploy the same reproducible application across environments.

ARCHITECTURE PRINCIPLE

More agents create more interfaces—not automatically more intelligence.

Split responsibilities only when the new boundary improves ownership, permissions, evaluation, or reliability. Every handoff must earn its cost.

What You Will Build

Reliable Core

- A typed Gemini triage agent
- Validated tools and recoverable errors
- Session state and searchable knowledge

Agent Team

- Three least-privilege specialists
- Coordinator delegation
- A deterministic resolution workflow

Production Edge

- MCP capability discovery
- Guardrails and observability
- Tests and container packaging

How Every Lab Progresses

1

Understand the architectural idea and the trade-off it addresses.

2

Inspect the starting checkpoint and predict the behaviour before running it.

3

Build one bounded capability using synthetic data and placeholder secrets.

4

Verify the result with a command, expected output, or visible event trace.

5

Record the checkpoint so the next lab can extend the same SupportOps system.

TOPIC 01

Single Agent Foundations

ADK architecture · Agent lifecycle · Gemini configuration · Structured I/O · Sessions

Key Concepts — Single Agent Foundations

1

ADK application anatomy

An ADK app exposes a `root_agent`; a Runner coordinates model calls, tools, events, sessions, and the final response.

2

Lifecycle

A user message enters a session, the agent reasons with Gemini, optional tools execute, events record the work, and a response returns.

3

Root agent contract

The `root_agent` is the discoverable entry point used by adk web, adk run, the API server, and deployment targets.

4

Instruction versus description

Instruction governs behaviour; description advertises capability so a coordinator can choose the right specialist.

5

Model selection

Pin a stable model for reproducibility. Use a latest alias for exploration only when automatic upgrades are acceptable.

6

Structured I/O

Pydantic schemas turn free-form model output into validated fields that code, tools, and later workflow nodes can consume.

Key Concepts — Single Agent Foundations (cont.)

7

Control nondeterminism

Narrow scope, explicit policies, schemas, examples, and tests reduce variation without pretending an LLM is deterministic.

8

Events

Events are the audit trail of messages, model responses, tool calls, tool results, state changes, and workflow progress.

9

Session

A session is one conversation thread identified by app, user, and session IDs; it owns ordered events and temporary state.

10

Stateless or stateful

Stateless calls are easier to scale; stateful interactions are needed when later turns depend on earlier choices or progress.

11

Worked example

SupportOps converts a vague ticket into category, urgency, summary, and next_action—then stores the typed result for the next turn.

12

When one agent is enough

Prefer one focused agent while one instruction set and one tool boundary remain understandable; split only when roles truly diverge.

Hands-On Labs — Single Agent Foundations

Lab 1

- Create the SupportOps Agent

Lab 2

- Add Structured Triage and Sessions

Create the SupportOps Agent

LAB 1

You create an isolated Python environment, configure a placeholder-based Gemini connection, inspect the ADK project contract, and run a minimal SupportOps agent in the ADK development web UI. The finished checkpoint proves the full user → Runner → agent → model → event → response path before tools, memory, or delegation add complexity.

You'll build

A clean work/ project containing a discoverable `support_ops.root_agent` and a captured first-turn event trace.

Tools: Python 3.10+, Google ADK 2.5.0, Gemini 3.6 Flash, ADK Web

Create the SupportOps Agent

STEP 1 OF 1



Run the smallest complete path first: expose root_agent, start ADK Web, send one synthetic ticket, then inspect the event trace before adding more moving parts.

```
$ adk web work
```

Create the SupportOps Agent

Test it

Run the portable pytest wrapper from Step 6 and expect all tests to pass. In ADK Web, the prompt must return a helpful support-oriented response, and the event panel must show one user message followed by a model-generated response with no tool call. Confirm work/CHECKPOINT-LAB-01.txt exists.

Add Structured Triage and Sessions

LAB 2

You replace free-form triage with a Pydantic contract, store the latest validated result through `output_key`, and inspect how two turns share one session. The lab demonstrates why structured output is an interface, not merely a formatting preference, and why an in-memory development session should not be mistaken for durable production storage.

You'll build

A typed `TicketTriage` response with `category`, `urgency`, `summary`, and `next_action` stored as `latest_triage` in the active session.

Tools: Google ADK, Pydantic, ADK Web session and event inspector, pytest

Add Structured Triage and Sessions

STEP 1 OF 1

1

Treat the model output as an API: validate four bounded fields, store the result under `latest_triage`, and inspect both the event history and the session state.

```
$ output_schema=TicketTriage, output_key='latest_triage'
```

Add Structured Triage and Sessions

Test it

From the repository root, the portable pytest command in Step 4 must pass. A billing prompt must return schema-valid JSON whose category is billing, and the ADK state panel must contain `latest_triage`. A second turn in the same session should use prior context; the same turn in a new session should require clarification.

Recap — Single Agent Foundations

1 You can now: LO1: Explain the ADK lifecycle and configure a focused Gemini root agent.

2 You can now: LO1, LO3: Validate structured agent output and distinguish session events from session state.

TOPIC 02

Tools, Memory and Sessions

Function calling · External APIs · Tool errors · Session state · Long-term memory · Vector retrieval

Key Concepts — Tools, Memory and Sessions

1

Tool contract

A tool has a name, purpose, typed arguments, documented constraints, and a structured result the model can interpret.

2

Schemas from Python

ADK inspects function names, type hints, defaults, and docstrings; ambiguous signatures create ambiguous tool calls.

3

Execution loop

Gemini selects a tool, ADK validates arguments, application code executes it, and the result returns to the model for the next decision.

4

Read versus write tools

Queries are lower risk. Mutating tools need authorisation, confirmation, idempotency, and a record of what changed.

5

Secrets

Credentials belong in environment variables or a managed secret store—never source code, prompts, logs, screenshots, or tool results.

6

External API reliability

Set timeouts, validate status and payload, retry only safe failures, respect rate limits, and preserve correlation IDs.

Key Concepts — Tools, Memory and Sessions (cont.)

7

Errors as data

Return a stable status and recoverable message so the agent can explain, retry safely, or ask for missing information.

8

Tool chaining

Let each tool do one job and pass compact results forward; long chains multiply latency, cost, permissions, and failure modes.

9

State scopes

Session state tracks one thread; user- and app-scoped prefixes support wider persistence when the selected service stores them.

10

Memory versus history

History records what happened; state holds current working facts; memory is searchable knowledge across sessions or sources.

11

Vector retrieval

Chunk documents, create embeddings, store vectors with metadata, retrieve top matches, then ground the response in returned evidence.

12

Use memory deliberately

Store stable preferences and reusable facts, not every utterance. Apply retention, consent, access, and deletion rules.

Hands-On Labs — Tools, Memory and Sessions

Lab 3

- Connect Reliable Function Tools

Lab 4

- Add Searchable Knowledge and Memory

Connect Reliable Function Tools

LAB 3

You run a local synthetic support API, wrap it with ADK function tools, and give the agent a deliberately small capability surface. The read path validates identifiers and normalises HTTP failures; the write path requires an idempotency key and explicit user intent. This creates a tool contract the model can reason about without receiving credentials or raw exceptions.

You'll build

A tool-enabled SupportOps agent that can retrieve a synthetic ticket and safely change its priority through a local HTTP API.

Tools: Google ADK Function Tools, FastAPI, Uvicorn, HTTPX, pytest

Connect Reliable Function Tools

STEP 1 OF 1

1

Separate the model from the HTTP boundary: validate first, call with a timeout, return stable status data, and make writes idempotent and explicitly authorised.

\$ get_ticket → HTTP API → structured result; update_ticket_priority → idempotency key → audited change

Connect Reliable Function Tools

Test it

From the repository root, the portable pytest command must pass. With the local API running, the agent must retrieve TKT-1001, change its priority to medium, and return `repeated=true` when the same idempotency key is reused. With the API stopped, the agent must report unavailability without inventing fields.

Add Searchable Knowledge and Memory

LAB 4

You index synthetic runbook passages in an in-process Chroma vector collection, expose a bounded search tool, and add preference tools backed by ADK ToolContext state. The agent can now ground technical guidance in retrieved evidence and remember a response-style preference without storing every conversation turn. You then inspect what survives a new session and what disappears after the development process restarts.

You'll build

A SupportOps agent with semantic runbook retrieval, source identifiers, and a user-scoped response-style preference.

Tools: Chroma 1.5.9, Google ADK ToolContext, session state, pytest

Add Searchable Knowledge and Memory

STEP 1 OF 1

1

Use the right context store for the job: vector retrieval for reusable runbook evidence, user-scoped state for one stable preference, and events for the immutable interaction history.

```
$ query → embedding → Chroma top matches → source-aware answer; ToolContext → user:response_style
```

Add Searchable Knowledge and Memory

Test it

From the repository root, the portable pytest command must pass. A duplicate-charge query must invoke `search_knowledge` and cite `RB-BILL-01`. After `remember_response_style` stores detailed, the same user should retrieve that value in a new session while the development service remains running.

Recap — Tools, Memory and Sessions

1

You can now: LO2: Build typed read and write tools around an external API with validation, timeouts, idempotency, and recoverable errors.

2

You can now: LO3: Implement vector-backed knowledge retrieval and user-scoped state while keeping history, state, and memory conceptually separate.

TOPIC 03

Multi-Agent Architecture

Specialisation · Coordinator and hierarchy · Delegation · Shared context · Graph workflows

Key Concepts — Multi-Agent Architecture

1

Why multiple agents

Split a system when distinct roles need different instructions, tools, permissions, models, ownership, or evaluation criteria.

2

Specialisation

A specialist owns a narrow outcome and small tool surface. Clear boundaries improve reasoning, testing, and least privilege.

3

Coordinator pattern

A coordinator receives the request, selects a specialist, supplies context, checks the result, and communicates one coherent answer.

4

Delegation signal

ADK uses specialist names, descriptions, modes, and coordinator instructions to decide when and how work should be delegated.

5

Supervisor and hierarchy

A supervisor governs several specialists; deeper hierarchies can scale ownership but add latency and harder failure analysis.

6

LLM-based routing

Flexible for ambiguous language, but probabilistic. Use bounded labels, examples, fallbacks, and telemetry around routing decisions.

Key Concepts — Multi-Agent Architecture (cont.)

7

Rule-based routing

Fast and predictable for explicit identifiers, permissions, or thresholds; brittle when language and intent are genuinely ambiguous.

8

Graph workflow

ADK 2.x Workflow edges make order and branches explicit, mixing LLM agents with deterministic functions and typed node outputs.

9

Pipeline pattern

A fixed classify → enrich → resolve → summarise sequence is appropriate when every request must pass the same controlled stages.

10

Shared invocation context

Collaborating agents can share session state, but keys and ownership must be documented to prevent accidental overwrites.

11

Handoff contract

Pass only the task, validated context, evidence, constraints, and expected output—not the coordinator's entire internal prompt.

12

Failure isolation

Set budgets, timeouts, fallbacks, and escalation paths per specialist so one failing capability does not collapse the whole team.

Hands-On Labs — Multi-Agent Architecture

Lab 5

- Build a Specialist Agent Team

Lab 6

- Orchestrate a Deterministic Resolution Pipeline

Build a Specialist Agent Team

LAB 5

You split SupportOps into three least-privilege specialists and place them behind one coordinator. Each specialist has a narrow description, instruction, mode, and tool set; the coordinator owns user communication and delegation. You test obvious, ambiguous, and out-of-scope requests to see how model-driven routing behaves and where descriptions become part of the architecture.

You'll build

A collaborative ADK agent team whose coordinator delegates synthetic requests to billing, technical, or account specialists.

Tools: Google ADK collaborative agents, Gemini 3.6 Flash, specialist tools, ADK trace inspector

Build a Specialist Agent Team

STEP 1 OF 1

1

Make delegation legible: each specialist needs a distinct description, one responsibility, the smallest tool set, and automatic return to the coordinator after one bounded task.

```
$ coordinator → billing | technical | account → coordinator → user
```

Build a Specialist Agent Team

✓ Test it

From the repository root, the portable pytest command must pass. In ADK Web, the three obvious prompts must delegate to `billing_specialist`, `technical_specialist`, and `account_specialist` respectively. The technical and account traces must not contain `get_ticket`. The out-of-scope prompt must not be forced into an unrelated specialist.

Orchestrate a Deterministic Resolution Pipeline

LAB 6

You create a graph-based workflow for requests that must follow an explicit process. A typed triage agent emits one bounded category; a deterministic router converts that value into a graph route; and one single-turn resolver produces the final bounded guidance. You compare this reliable process with the flexible coordinator from Lab 5 and document when each pattern is appropriate.

You'll build

A SupportOps Workflow with explicit START → triage → route → specialist edges for billing, technical, account, and general requests.

Tools: Google ADK 2.x Workflow, Event routing, Pydantic schemas, Gemini specialist nodes, pytest

Orchestrate a Deterministic Resolution Pipeline

STEP 1 OF 1



Use the graph when process control matters: typed triage creates one route label, a deterministic function emits that route, and only the matching resolver runs.

```
$ START → triage_agent → route_triage → {billing | technical | account | general}
```

Orchestrate a Deterministic Resolution Pipeline

Test it

From the repository root, the portable pytest command must pass. Each clear prompt must follow triage_node → route_triage → exactly one matching resolver, with the chosen category visible in the triage output. No ticket or knowledge tool should run in this controlled graph checkpoint because the resolver contract explicitly forbids claiming external work.

Recap — Multi-Agent Architecture

1

You can now: LO4: Design a coordinator with bounded billing, technical, and account specialists using clear delegation contracts.

2

You can now: LO4, LO5: Build an ADK graph workflow that constrains classification, routing, and specialist execution through typed edges.

TOPIC 04

MCP and Production Design

MCP discovery · Structured context · Routing · Observability · Guardrails · Deployment and scale

Key Concepts — MCP and Production Design

1

MCP client and server

An MCP server publishes capabilities; an ADK McpToolset acts as a client and adapts discovered tools for an agent.

2

Protocol primitives

Tools perform actions, resources expose data, and prompts provide reusable interaction templates through declared schemas.

3

Capability discovery

The client lists server tools at runtime, reads their schemas, and exposes only the approved subset to the model.

4

Structured context

Typed arguments, results, metadata, identity, and correlation IDs create an inspectable context envelope between components.

5

Least-privilege filtering

Use tool filters, read-only modes, scoped credentials, and separate servers so an agent sees only what its role needs.

6

Connection lifecycle

Local stdio and remote HTTP transports have different scaling, authentication, timeout, and shutdown requirements.

Key Concepts — MCP and Production Design (cont.)

7

Routing strategy

Combine deterministic rules for policy with model routing for ambiguity; record the chosen route and why it was selected.

8

Observability

Correlate logs, traces, metrics, events, model calls, tool calls, latency, token use, errors, and outcomes across one request.

9

Evaluation

Test final answers, route choice, tool trajectories, safety behaviour, latency, and cost with representative multi-turn cases.

10

Guardrail layers

Validate input, authorise tools, constrain arguments, inspect results, filter output, and require human approval for high-impact actions.

11

Threat model

Plan for prompt injection, data exfiltration, excessive agency, confused deputy risks, poisoned memory, and unsafe tool chaining.

12

Deployment and scale

Externalise state, package dependencies, add health checks, cap concurrency, manage quotas, and choose Agent Runtime, Cloud Run, GKE, or another container host.

Hands-On Labs — MCP and Production Design

Lab 7

- Discover Tools Through MCP

Lab 8

- Harden, Observe, and Package SupportOps

Discover Tools Through MCP

LAB 7

You move runbook lookup behind a local Model Context Protocol server and connect an ADK agent through McpToolset over stdio. The agent discovers the server's schema at runtime, while `tool_filter` limits the exposed capability to one approved read-only tool. You inspect discovery, invocation, source-labelled results, connection lifecycle, and the trust boundary between server data and agent instructions.

You'll build

A local SupportOps MCP server plus an ADK client agent that discovers and calls only `lookup_runbook`.

Tools: Model Context Protocol Python SDK, FastMCP, ADK McpToolset, stdio transport, ADK trace inspector

Discover Tools Through MCP

STEP 1 OF 1

1

Keep the boundary explicit: the MCP server declares a read-only schema, `McpToolset` discovers it over `stdio`, and `tool_filter` exposes only `lookup_runbook` to Gemini.

```
$ MCP server:list_tools → McpToolset adapter → approved ADK tool → call_tool → source-labelled result
```

Discover Tools Through MCP

✓ Test it

From the repository root, the portable pytest command must pass. In ADK Web, a technical query must invoke the discovered `lookup_runbook` tool, return source `MCP-RB-TECH-02`, and expose no other MCP capability. An unsupported topic must produce the bounded allowed-topics error.

Harden, Observe, and Package SupportOps

LAB 8

You apply an input callback before Gemini, a tool-argument callback before MCP execution, and structured logs that avoid prompt or secret content. You reintegrate the specialist team and guarded MCP specialist beneath one production supervisor, build deterministic policy tests and an executable evaluation catalogue, then package the application behind the ADK API server in a non-root container. The final review covers state externalisation, managed secrets, health checks, quotas, timeouts, deployment choices, and operational ownership.

You'll build

A production-oriented multi-agent SupportOps supervisor with the specialist hierarchy, tested policy callbacks, bounded MCP access, structured telemetry, executable evaluation cases, and a Docker image definition.

Tools: ADK callbacks, Python logging, pytest, ADK API server, Docker, OpenTelemetry-ready configuration

Harden, Observe, and Package SupportOps

STEP 1 OF 1

1

Layer production controls around the agent loop: validate before the model, authorise before the tool, record metadata without sensitive content, test policy outcomes, then package the same versioned app.

```
$ input guard → model → tool guard → MCP → result → trace/log → API server → container
```

Harden, Observe, and Package SupportOps

✓ Test it

From the repository root, the portable pytest command must pass and retain evidence/pytest.xml; test_production must show both the nested specialist team and guarded MCP specialist. The allowed technical prompt must call lookup_runbook with topic=technical; payroll must be blocked by the tool policy; the sensitive request must be answered by guard_model_input without a model or tool call. If Docker is available, support-ops:c829-v1.0 must build and expose the ADK API documentation on port 8080.

Recap — MCP and Production Design

1

You can now: LO5: Expose a bounded capability through an MCP server and connect it to ADK with filtered runtime discovery.

2

You can now: LO6: Add layered guardrails, privacy-aware telemetry, evaluation cases, and reproducible container packaging.



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Agent Foundations

Explain the ADK agent lifecycle and configure a Gemini agent with clear instructions, typed input or output, and session-aware behaviour.

2

Reliable Tools

Build reliable function tools and API integrations with explicit schemas, validation, error handling, and safe secret management.

3

State & Memory

Use session state and searchable memory to preserve relevant context without confusing conversation history, working state, and long-term knowledge.

4

Agent Teams

Design multi-agent systems with specialist roles, coordinator delegation, shared context, and deterministic graph workflows where control matters.

5

MCP & Routing

Connect ADK agents to MCP capabilities and choose an appropriate routing strategy for tools, specialists, and workflow branches.

6

Production Readiness

Apply evaluation, observability, security guardrails, and deployment practices to prepare a multi-agent application for production use.

Keep Practising After the Course

1

Re-run the labs on your own machine — repetition is what makes the skills stick.

2

Apply the techniques to a real process or project in your own organisation.

3

Your course material stays available at <https://lms-tms.tertiaryinfotech.com/>.

4

Explore the follow-on courses at www.tertiarycourses.com.sg.

THANK YOU FOR ATTENDING

Thank You!

You now have the practical skills from every lab in this course — keep applying them to your own work.